

Simple Study Group Finder Web Application

Ishika Bhuva

22.05.2026

Project Goal & Problem Statement

Problem

- Students struggle to:
 - Find study partners in their courses (de Bie (2022))
 - Organize collaborative learning
 - Track academic study groups

Goal

- Develop a web-based platform that:
 - Secure authentication with JWT
 - Persistent MongoDB storage
 - Real-time group messaging
 - Admin moderation support

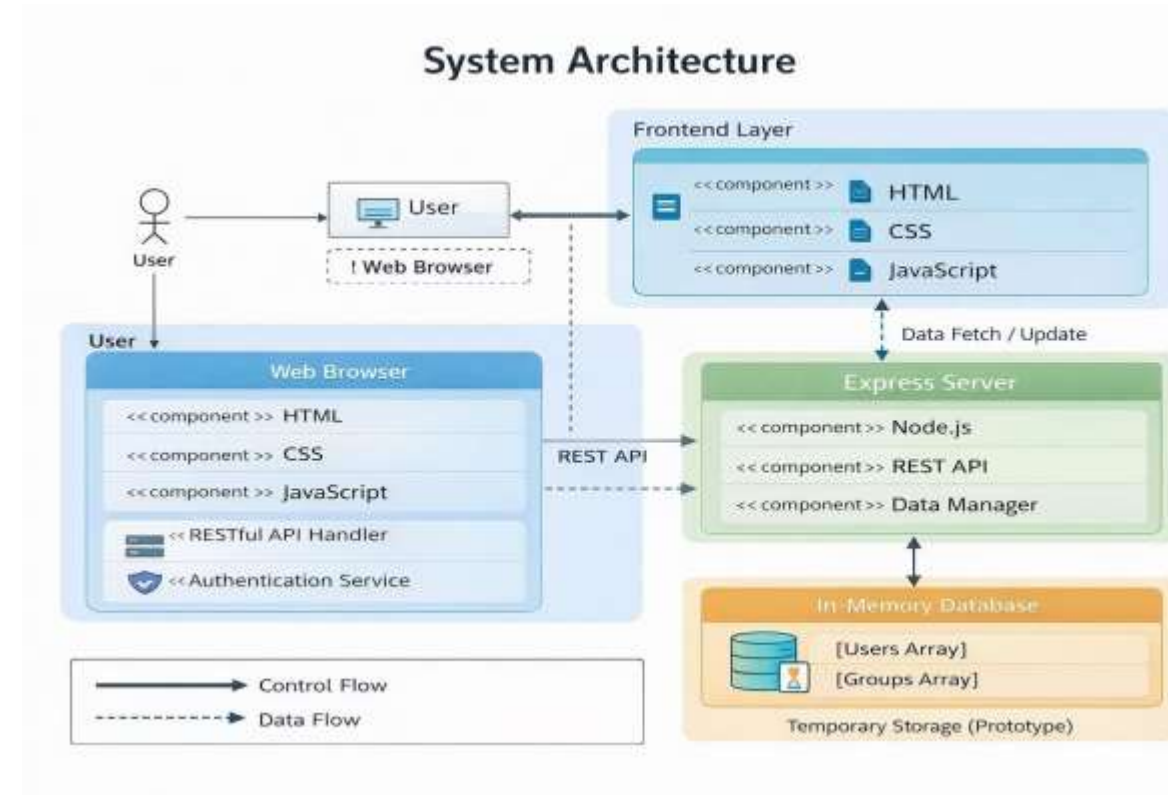
System Architecture

Frontend

- HTML (Structure)
- CSS (UI Styling)
- Vanilla JavaScript (State Management & UI Rendering)

Backend

- Node.js
- Express Framework
- REST API Endpoints (Herdiyatmoko, 2022)
- MongoDB + Mongoose (persistent storage & schema validation)
- WebSocket / Socket.IO communication



Modules / Building Blocks

Component Architecture Mapping



Architecture Layer	Component	Responsibility
Frontend	UI Renderer	Displays dashboard, groups, and updates interface dynamically
Frontend	State Manager	Handles user input, course selection, and UI state updates
Frontend	API Handler	Sends HTTP requests to backend endpoints and processes responses
Backend	Express Server	Handles routing, processes requests, and manages centralized error handling via global middleware.
Backend	Authentication Service	Validates user login and manages JWT authentication, password hashing and route protection
Backend	Group Service	Creates, retrieves, and manages study groups
Backend	Course Service	Handles course addition and removal operations
Data Layer	MongoDB Database	Temporarily stores users and groups using arrays
Data Layer	Mongoose Data Layer	Performs validated CRUD operations on MongoDB collections

Frontend Design & UI

UI Structure

- Sidebar Navigation (Discover / My Groups / Chats)
- Dashboard Statistics Cards
- Course Filter Ribbon
- Group Card Grid Layout
- Modal Windows (Create Group / Add Course)
- Real-time chat interface
- JWT-protected dashboard access

Simple Study Group Finder Web App

- Discover
- My Groups
- Chats

MY ENROLLED COURSES

No courses tracked

Daniel
TU

Discover Groups

Find study partners matching your courses.

Track New Course

+ Start Group

MARKETPLACE

4

ENROLLED

0

PENDING

0

JOINED

1

All Groups

Explore Groups

Search by name...

CS201

Data Structures Study

Weekly review of Trees and Graphs.



Joined

ECON10

Econ Paper Prep

Analyzing global trade trends.



Join

MED400

MCAT Flashcards

Intense daily quiz sessions.



Join

ENGL302

Modern Lit Discussion

Post-modern novel analysis.

Backend Implementation

Key Engineering Implementation:

- **Data Integrity & Sanitization:**
 - Explicitly casting groupId to a Number type to prevent logic failures during resource lookup.
- **Concurrency Constraints:**
 - Server-side enforcement of group limits (members $\geq$ limit) to maintain data consistency and prevent over-enrollment.
- **Fault Tolerance:**
 - Implemented **Global Middleware** for centralized exception handling, ensuring the application remains stable and returns professional JSON error responses instead of crashing.
- **Persistent Storage:**
 - MongoDB collections retain users, groups and messages after server restart.
- **Security:**
 - JWT authentication and bcrypt password hashing implemented for protected routes.

```
app.post('/groups/join', (req, res) => {
  const { userId, groupId } = req.body;

  // High-level: Input Sanitization
  const targetGroupId = Number(groupId);
  const user = users.find(u => u.id === userId);
  const group = groups.find(g => g.id === targetGroupId || g.id === groupId);

  if (!user || !group) {
    return res.status(404).json({ error: "Transaction Failed: Target user or group resource not found" });
  }

  if (user.joinedGroups.includes(groupId)) {
    return res.json(user);
  }

  // Business Logic: Capacity Validation
  if (group.members >= group.limit) {
    return res.status(400).json({ error: "Concurrency Limit: This study group has reached maximum capacity" });
  }

  user.joinedGroups.push(groupId);
  user.requests = user.requests.filter(r => r !== groupId);
  group.members += 1;

  res.json(user);
});
```

```
app.use((err, req, res, next) => {
  console.error(`[Internal Server Error]: ${err.message}`);
  res.status(500).json({
    error: "System Exception",
    message: "An unexpected error occurred within the application layer."
  });
});

app.listen(PORT, () => {
  console.log(`[Server Initialized] Listening on http://localhost:${PORT}`);
});
```

Development Phase (Reflection)

Framework & Tool Setup

- Node.js runtime environment
- Express.js for routing
- Visual Studio Code
- Git (Version control recommended) (Ponuthorai & Loeliger 2022)
- Browser DevTools for debugging

Third-Party Libraries

- Express
- Body-parser
- CORS

Rationale

- Lightweight
- Easy REST implementation
- Fast development cycle

Scrum Technology

Scrum sprint planning
Product Backlog Items (PBIs)
Sprint reviews and backlog refinement

Design Decisions & Rationale

Important Decisions

- **Single Page Application structure**
 - Faster user experience
 - No page reloads
- **MongoDB with Mongoose**
 - Simplifies testing
 - Persistent scalable storage with schema validation
- **REST API structure**
 - Clear separation of frontend & backend
 - Scalable for future database integration (Gowda & Gowda 2024)
- **Optimistic UI updates**
 - Immediate feedback for join actions
 - Improved user experience
- **WebSocket / Socket.IO**
 - Real-time messaging without constant polling

Testing & Risk Analysis

Test Data

Predefined groups:

Group Name	Course	Members	Limit
Data Structures Study	CS201	4	6
Econ Paper Prep	ECON10	2	5
MCAT Flashcards	MED400	9	10

Performed Tests

- Login validation (empty fields)
- Course addition & removal
- Group join validation
- Group capacity restriction
- UI state updates

Risks Identified

- Scalability under high user load
- JWT/session misuse
- Real-time communication failure
- Data validation and duplicate membership issues

Project Risks & Mitigation

Data Loss Risk

Risk of data inconsistency or database failure

Vulnerability Risk

Weak authentication mechanisms can expose user accounts and sensitive platform interactions
Scalability

Performance Risk

In-memory data storage limits system performance when handling large numbers of users

Risk Category	Description	Impact	Mitigation Strategy
Data Loss Risk	No database; all data stored temporarily in memory	High	Integrate MongoDB or SQL database for persistent and reliable data storage
Security Risk	Risk of token misuse or weak credential handling	High	Implement JWT authentication, password hashing, and HTTPS protocols
Scalability Risk	System cannot handle increased users or large datasets	Medium–High	Deploy scalable cloud infrastructure and replace in-memory storage with database
Performance Risk	API response time may degrade with increased load	Medium	Optimize backend logic and introduce caching mechanisms
Usability Risk	Lack of real-time chat reduces user engagement	Medium	Implement WebSockets for real-time communication functionality

System Requirements Specification

Functional Requirements

- System must support user registration, course tracking, group creation, joining, and messaging features

Non-Functional Requirements

- Response time below 500ms
- No more than 3 clicks for key tasks
- JWT expiry = 1 hour
- Browser compatibility targets

System Constraints

- No institutional SSO
- Limited admin moderation
- No cloud deployment yet
- No accessibility audit completed

Requirement Type	Requirement	Description
Functional	User Registration	Users must be able to create accounts and log into the platform securely
Functional	Course Management	Users can add or remove enrolled courses for personalised group discovery
Functional	Group Discovery	System should display available study groups based on selected courses
Functional	Group Creation	Users must be able to create new study groups with defined capacity limits
Functional	Group Joining	Users can join groups with validation checks such as maximum capacity
Functional	Messaging	Users should communicate within groups through a basic chat interface
Non-Functional	Performance	System should respond to user requests within acceptable time under normal load
Non-Functional	Scalability	Architecture should support future integration with databases and increased users
Non-Functional	Security	User data must be protected using authentication and secure communication protocols
Non-Functional	Usability	Interface should be simple, intuitive, and easy to navigate for all users
Non-Functional	Reliability	System should operate consistently without crashes during standard operations
Constraint	Storage Limitation	Data is stored temporarily in memory, not persisted across sessions
Constraint	Authentication Limitation	Basic login implemented without advanced security mechanisms like JWT
Constraint	Communication Limitation	Messaging is not real-time and lacks WebSocket-based implementation

Reflection & Future Improvements

Reflection on Development Phase

- Project goals achieved successfully
- Core features implemented and tested
- REST architecture working reliably
 - UI responsive and interactive
- Implemented environment-based configuration (process.env.PORT) for cloud-readiness.
- Introduced global middleware to handle unexpected system exceptions gracefully.

Improved Opportunities

- Institutional single sign-on
- Accessibility testing
- Cloud deployment
- Notification system
- Advanced moderation tools
- Automated testing pipelines (NANDAMURU, 2025)

Version Control & DevOps Standards

Repository Management:

- Transitioned from ZIP-based storage to a professional GitHub structure.

Clean Registry:

- Implemented .gitignore to exclude environment-specific folders (node_modules).
- Environment variable configuration using .env
- Modular backend structure (routes/controllers/models/services)

Documentation:

- Included a comprehensive README.md with installation and operational instructions.

References

- de Bie, A. (2022). Respectfully distrusting 'Students as Partners' practice in higher education: Applying a Mad politics of partnership. *Teaching in Higher Education*, 27(6), 717-737.
- Gowda, P., & Gowda, A. N. (2024). Best Practices in REST API Design for Enhanced Scalability and Security. *Journal of Artificial Intelligence, Machine Learning and Data Science*, 2(1), 827-830.
- Herdiyatmoko, H. F. (2022). Back-End System Design Based on Rest Api. *Jurnal Tekinkom (Teknik Informasi dan Komputer)*, 5(1), 123-129.
- NANDAMURU, A. P. (2025). REAL TIME WEB CHAT APPLICATION USING WEBSOCKETS IN DJANGO. *Journal of Nonlinear Analysis and Optimization*, 16(1).
- Ponuthorai, P. K., & Loeliger, J. (2022). *Version Control with Git*. " O'Reilly Media, Inc."